



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

软件体系结构

第12章 Kruchten的4+1模型

赵庆玲

南京理工大学计算机科学与工程学院



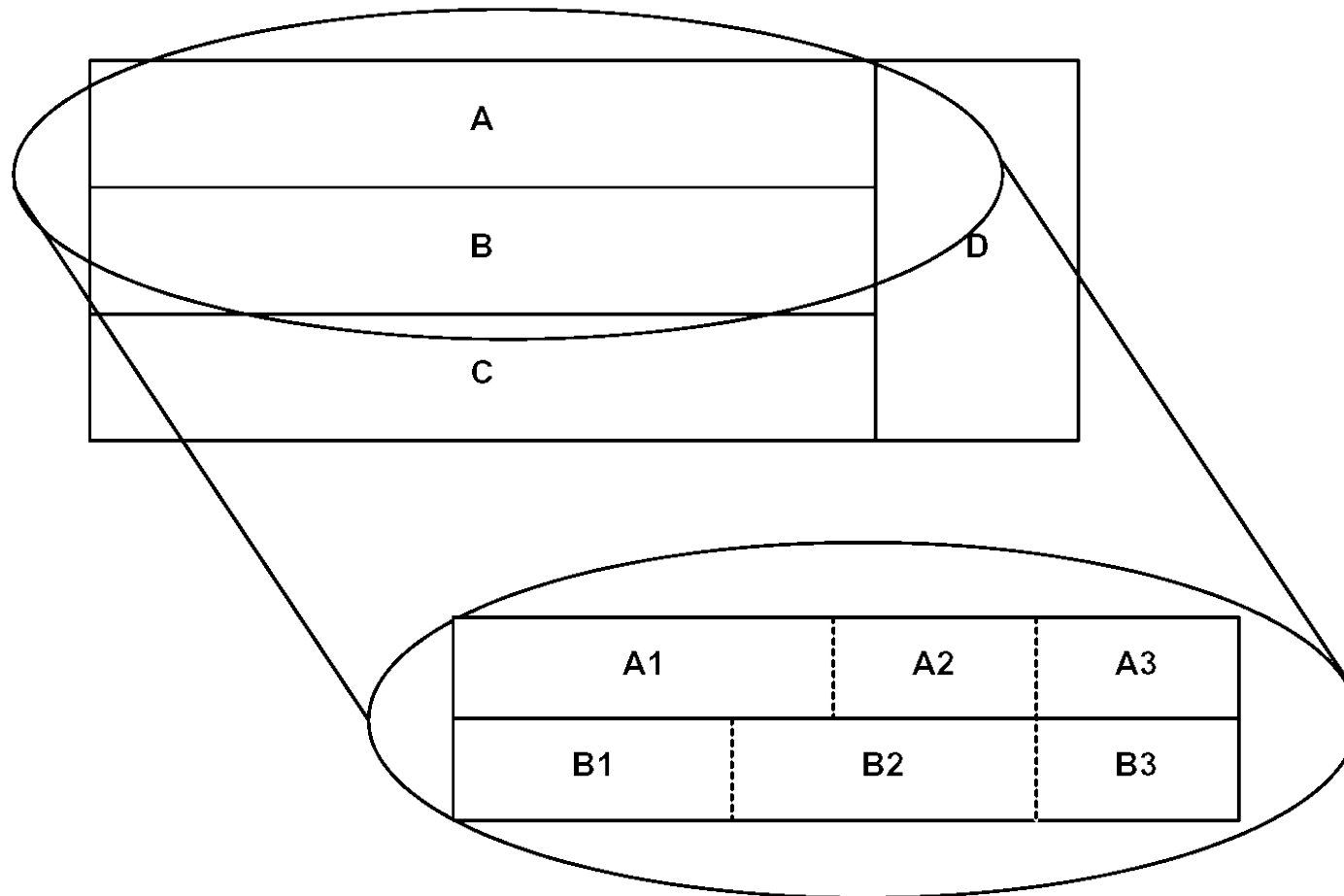
- 本章参考
Philippe Kruchten ——
《Architectural Blueprints—The “4+1”
View Model of Software Architecture》

假定你是Module Designer



- 你最近加盟一家公司，并被安排在一个新项目的开发组中。虽然你富有经验，但是对此项目所涉及的领域还是一个新手。系统的高层体系结构设计已经完成。
- 你的老板（项目经理）让你预计你将要完成的几个模块的开发时间。
- 你怎么办？

假定你是Module Designer



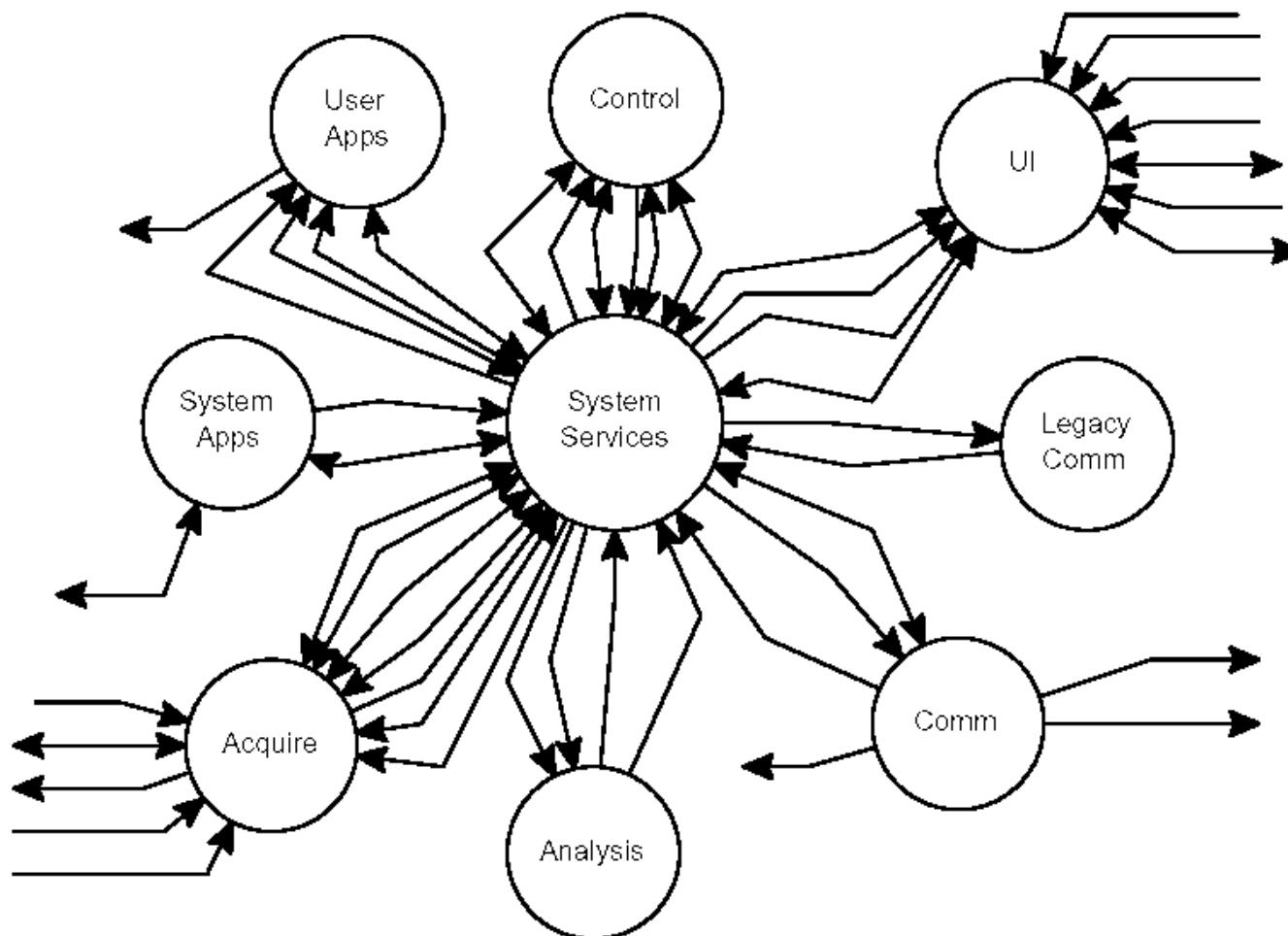
- 你来开发A2和A3，怎么开始？

假定你是Consultant(顾问)



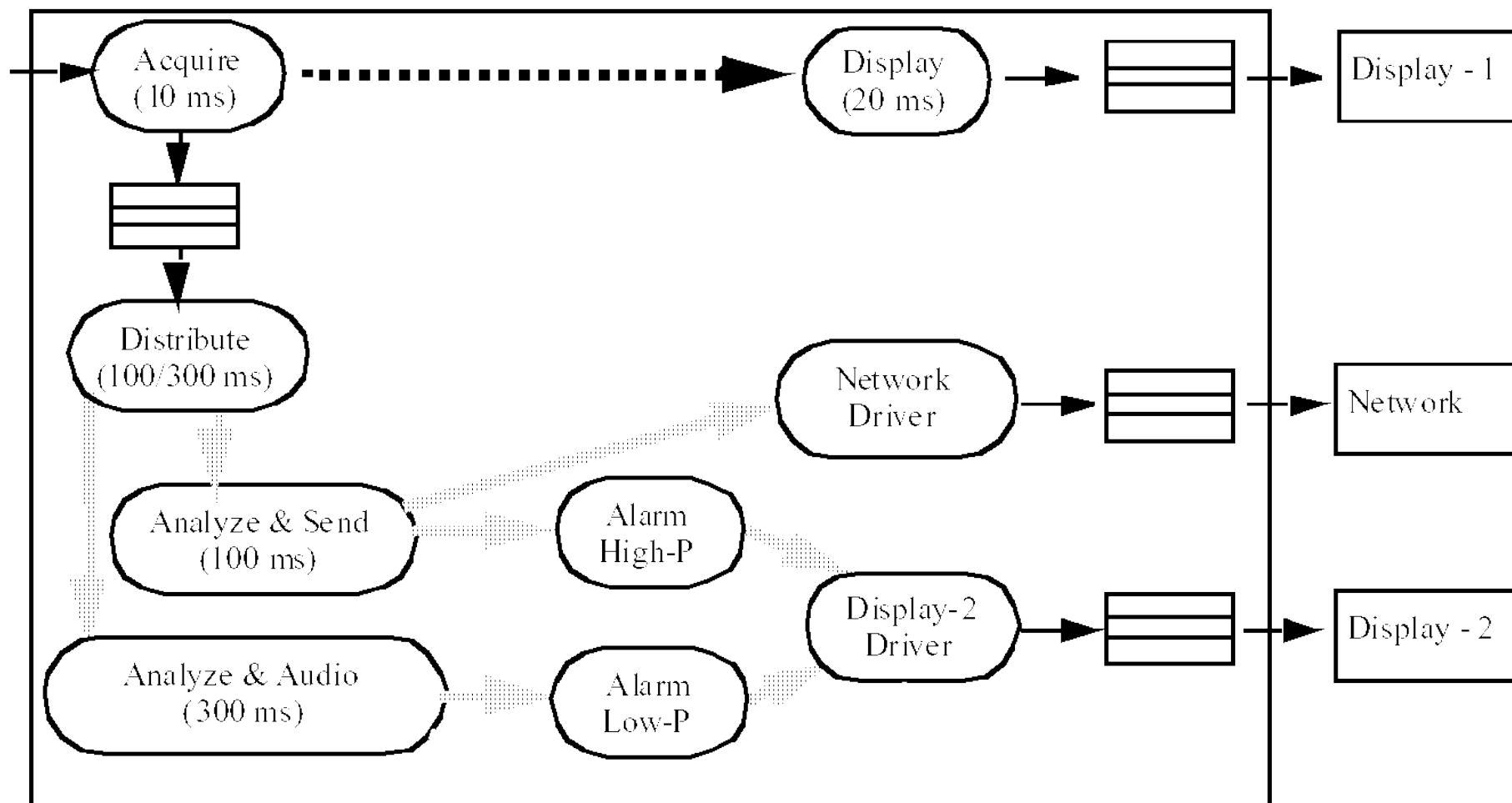
- 你是一个请来的顾问，对一个体系结构设计进行评估。Modifiability和Performance是重要的体系结构质量因素。
- 你会询问什么样的信息？

假定你是Consultant(顾问)



- 面对这样的图，你会有什么反应？

假定你是Consultant(顾问)



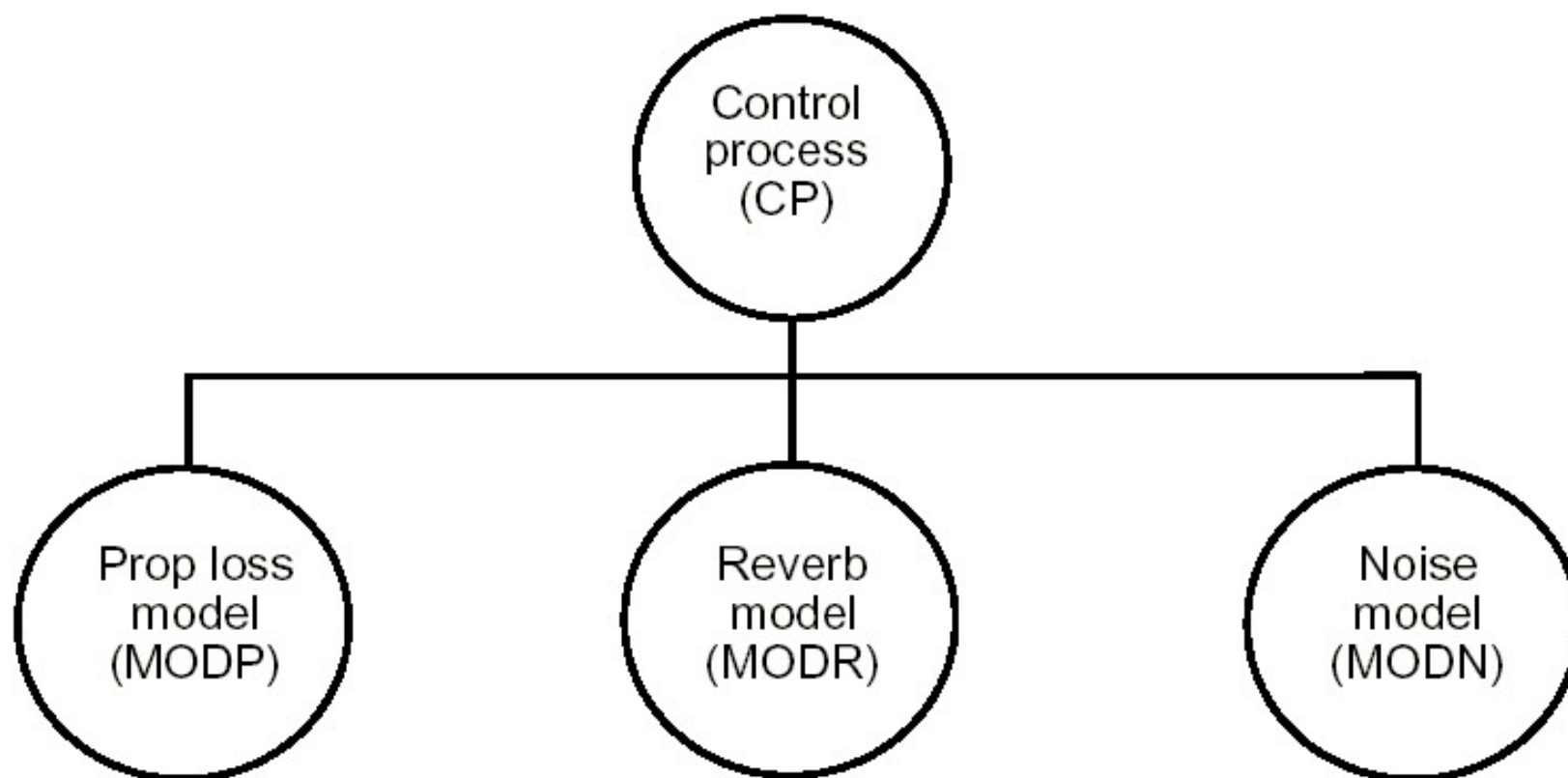
- 面对这样的图，你会有什么反应？

- 软件开发过程中各种角色之间交流设计思想的媒介
- 进行上层分析的基础。此基础上可以验证体系结构设计方案，精炼或改变必要的方案
- 让别人理解系统的第一手资料

- 基本想法是什么？
- 我该做什么 (如，实现哪些需求)？
- 我该在哪做 (如，这项功能实现在哪里)？
- 我和谁交互？接口是什么？
- 有什么可以重用的代码？
- 必须遵从什么约定(质量目标、旧体系/接口、预算等)？
- 有哪些硬性规定(设计、接口、约束等)？

- 体系结构的必要需求(driving requirement)是什么(如, performance, availability, security, modifiability, interoperability)?
- 各种体系结构视图是如何描述的?
 - 抽象出来什么?
 - 功能怎样分解?
 - 功能怎样分配?
 - 使用什么硬件以及软件怎样布置在硬件上?
- 采用了哪些体系结构风格?

这是什么？



- 很多事情没有说：
 - 组件类型
 - 连接件类型
 - 圆圈和箭头代表什么？
 - 这种布局的意义是什么？
 - 为什么**CP**要放在上层？
- 只画出方框和线条不是体系结构，只是体系结构的开始

好的体系结构描述的必要元素

- 需求陈述
 - 商业环境、产品的背景、领域
- 描述环境
 - 必须和什么系统交互、外部接口
- 使用体系结构图
 - 用恰当的线框
 - 简洁的说明

好的体系结构描述的必要元素

- 考虑实现时的限制
 - 但是仅在它们能影响体系结构设计范围内
 - 被限定的下层结构、处理器需求
 - 通常包含其他结构图
- 体系结构设计的原理
 - 它怎样去符合需求与约束
 - 其他的设计

- 风格/产品线问题
 - 设计可变的尺度
 - 体系结构的那个方面必须不被改变？
- 管理问题
 - 暗含开发团队的组织结构
 - 体系结构评审情况
- 其他设计问题
 - 代码重用、标准的运用
 - 风险分析
 - 运作、管理和维护

- 线和框有不同的形状/颜色，并有图例说明
- 用表格总结方案选择等等各种问题
- 图并不试图去表达很多信息：把信息分散到需要表达它的各个视图中
- 每个体系结构视图必须在一页内完成
- 清晰地区分出哪些是体系结构视图，哪些不是

- 所有的线看起来都一样
- 箭头不代表任何涵义
- 箭头代表很多涵义
- 实现与文档冲突
- 没有图例
- 太多的必要需求

- 系统需要多种视图来描述
 - 其中的一小部分是描述体系结构的
- 运行时视图/动态视图(组件和连接件)
 - 在高层分解成组件和连接件
- 代码视图
 - 模块关联和依赖
 - 使用/调用/和...共享数据
 - 文件和目录、工程和编译文件、版本控制
- 物理视图
 - 把计算单元分配到各个进程或处理器

- Philippe Kruchten, *Architectural Blueprints—The “4+1” View Model of Software Architecture*, IEEE Software 12 (6), 1995, pp. 42-50
- *Release 6A Segment/Design Specification for the ECS Project*, Section 4.4. NASA Report 305-CD-600-001, pages 4-160-185. March 2001
<http://edhs1.gsfc.nasa.gov/waisdata/toc/cd30560001toc.html>

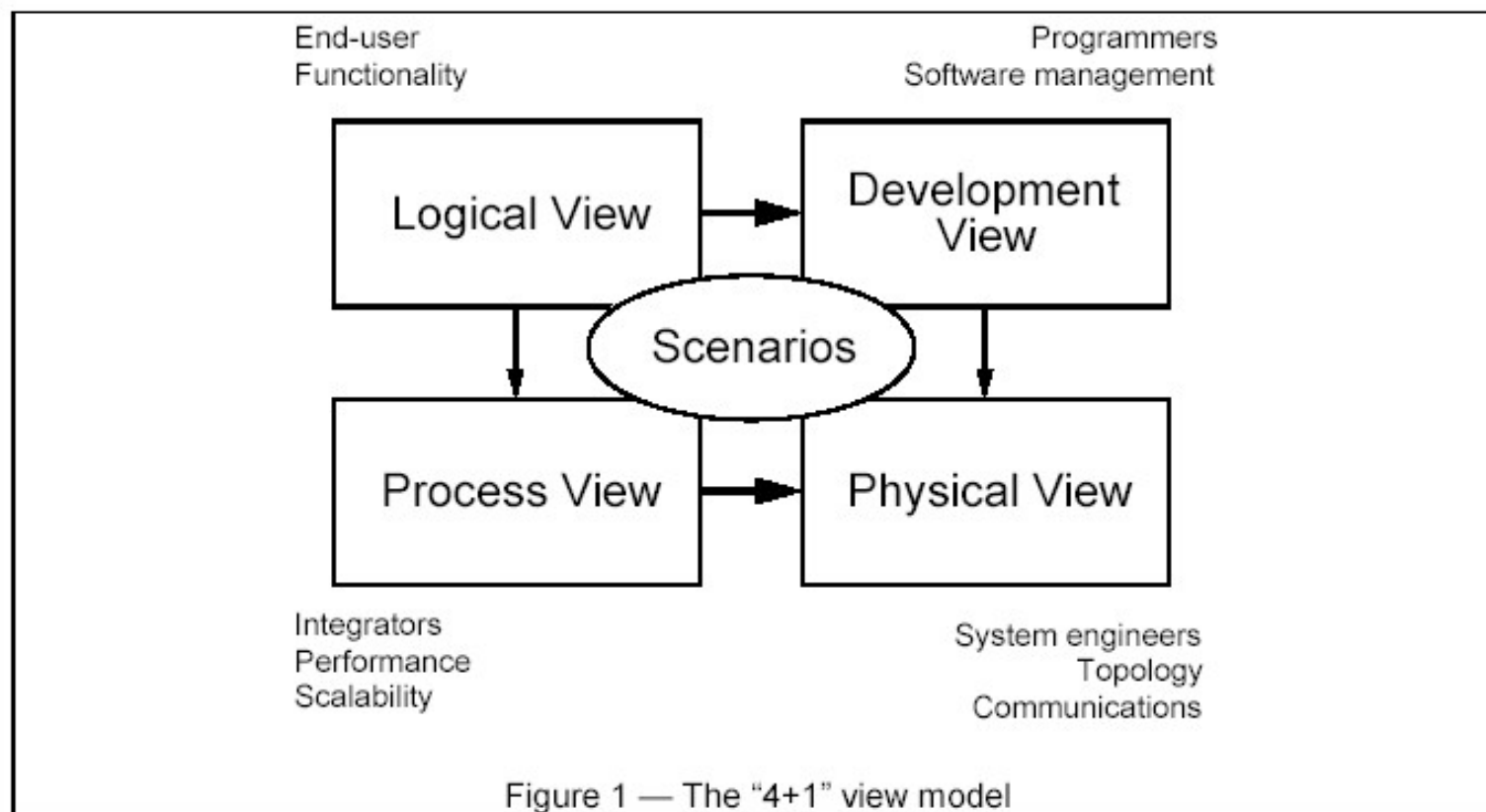


“4+1”模型

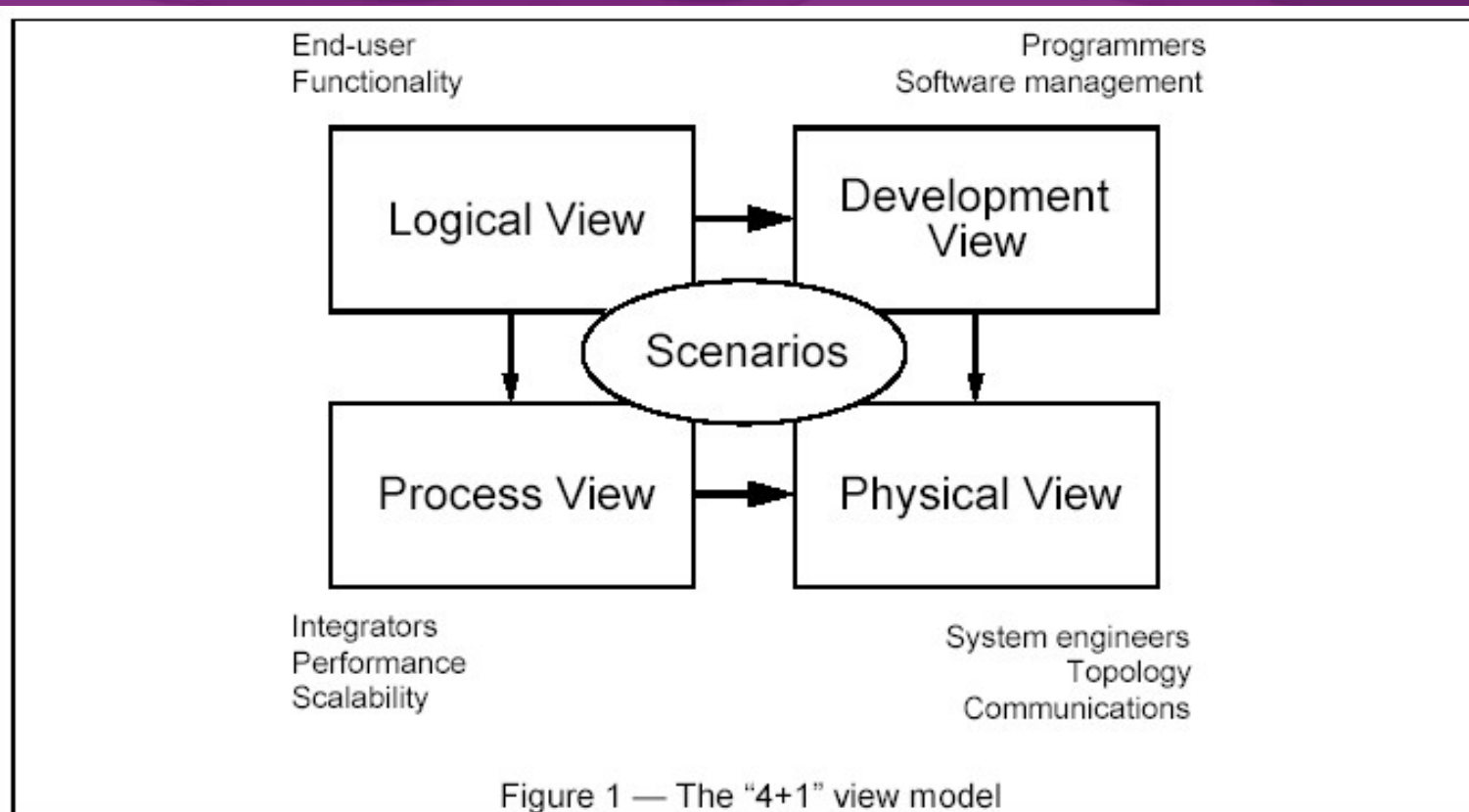
- Rational公司的Philippe Kruchten在1995年提出了用于体系结构描述的“4+1”模型。该模型建立在体系结构的Perry & Wolf定义和Berry Boehm定义的基础上。
- 该模型采用多视图模型的方法描述软件体系结构。为了最终能够处理富于挑战性的、大规模的软件系统，该模型由5个视图构成。

- ◆ 逻辑视图 当采用面向对象的设计方法时，逻辑视图即是对象模型。
- ◆ 过程视图 描述系统的并发和同步方面的设计。
- ◆ 物理视图 描述软件到硬件之间的映射关系，反映系统在分布方面的设计。
- ◆ 开发视图 描述软件在开发环境下的静态组织。

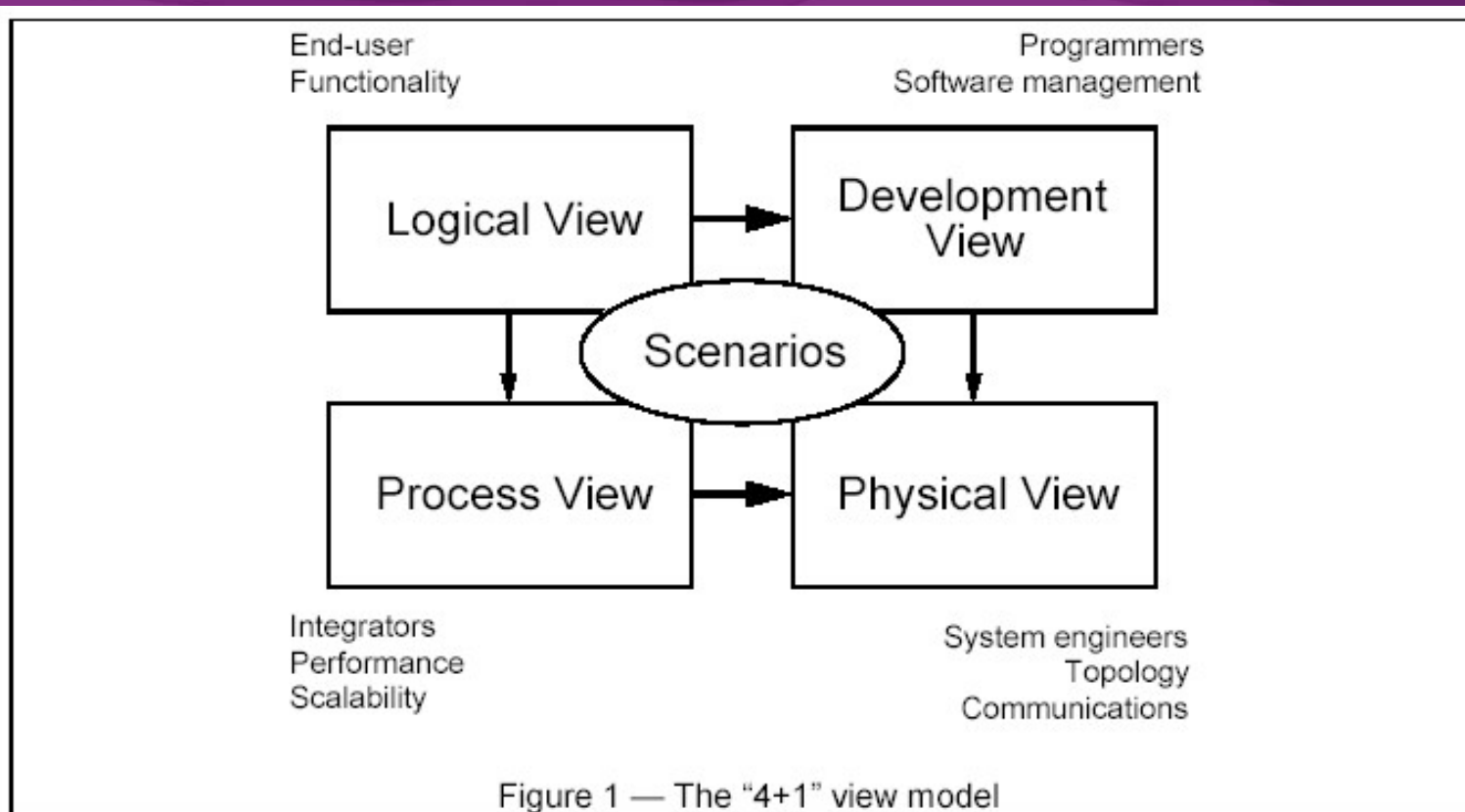
◆ 场景视图：汇总



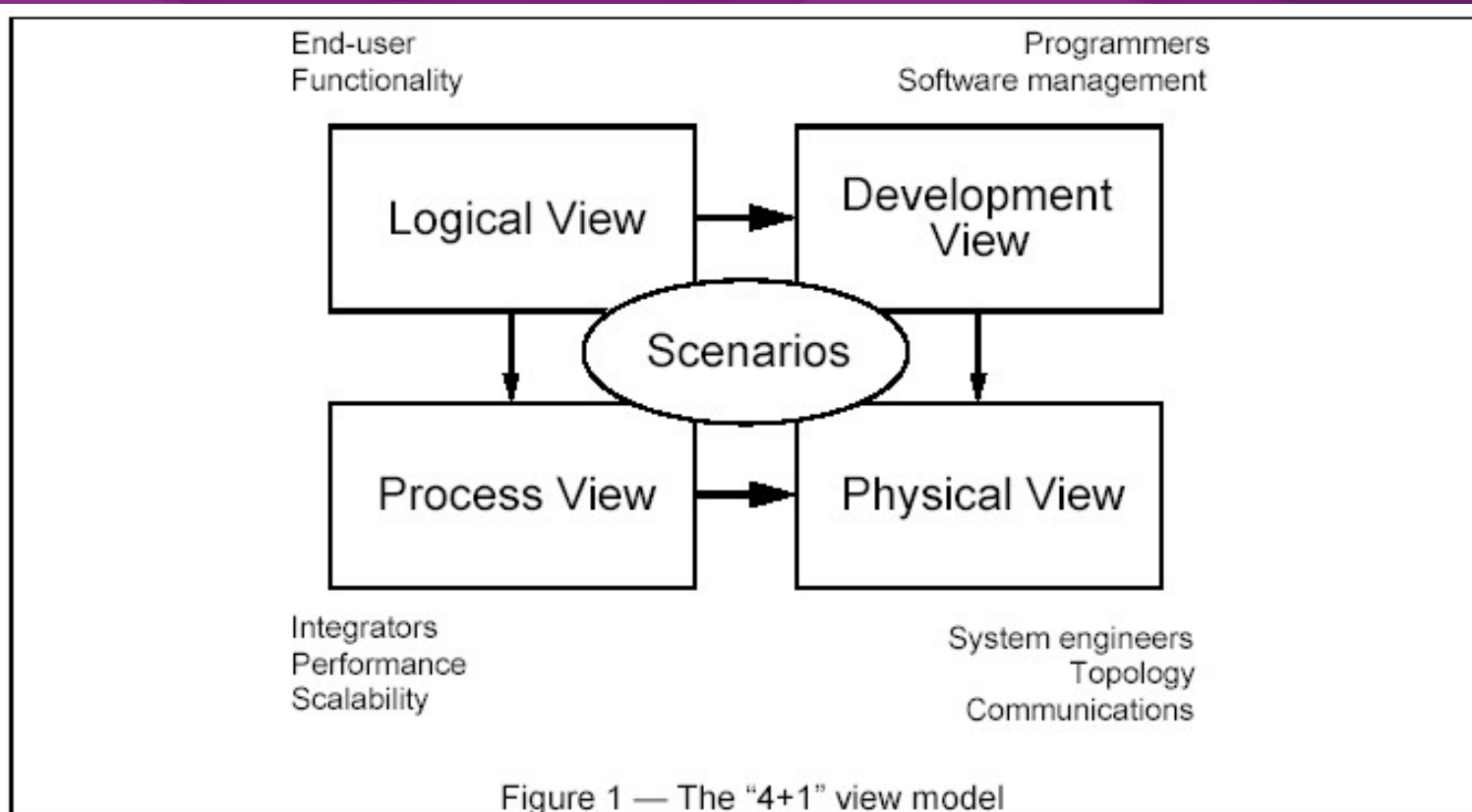
对体系结构进行的描述是围绕着以上4个视图展开的。然后，通过选择出的一些用例对体系结构加以说明。这些用例被称作场景(scenarios)，它们构成了第5个视图。实际上，体系结构在某种程度上是由场景演化而来的。



体系结构的概念在每个视图里面都可以独立应用。这就是说，可以在每个视图里面定义体系结构的各种组成元素，如构件、连接件等。对于不同的视图，还可以选择不同的体系结构风格，因此在同一个系统结构中使用多种风格。此外，在每一种视图里，我们使用该视图特定的符号。这避免了符号用法和意义的混乱。“4+1”视图模型是一个十分通用的模型：可以使用其他的符号表示法，也可以使用其他的设计方法，尤其是逻辑视图和过程视图的分解。



“4+1”模型实际上使得有不同需求的人员能够得到他们对于软件体系结构想要了解的东西。系统工程师先从物理视图，然后从过程视图靠近体系结构。最终使用者、客户、数据专家从逻辑视图看体系结构；项目经理、软件配置人员从开发视图看体系结构。



要指出的是，不是所有的软件体系结构都需要完整的“4+1”视图。没有用的视图在体系结构描述中可以被省略，例如对于非常小的系统，逻辑视图和开发视图有可能非常相似以至于没有必要把它们分开描述。场景视图在各种环境下都是有用的。

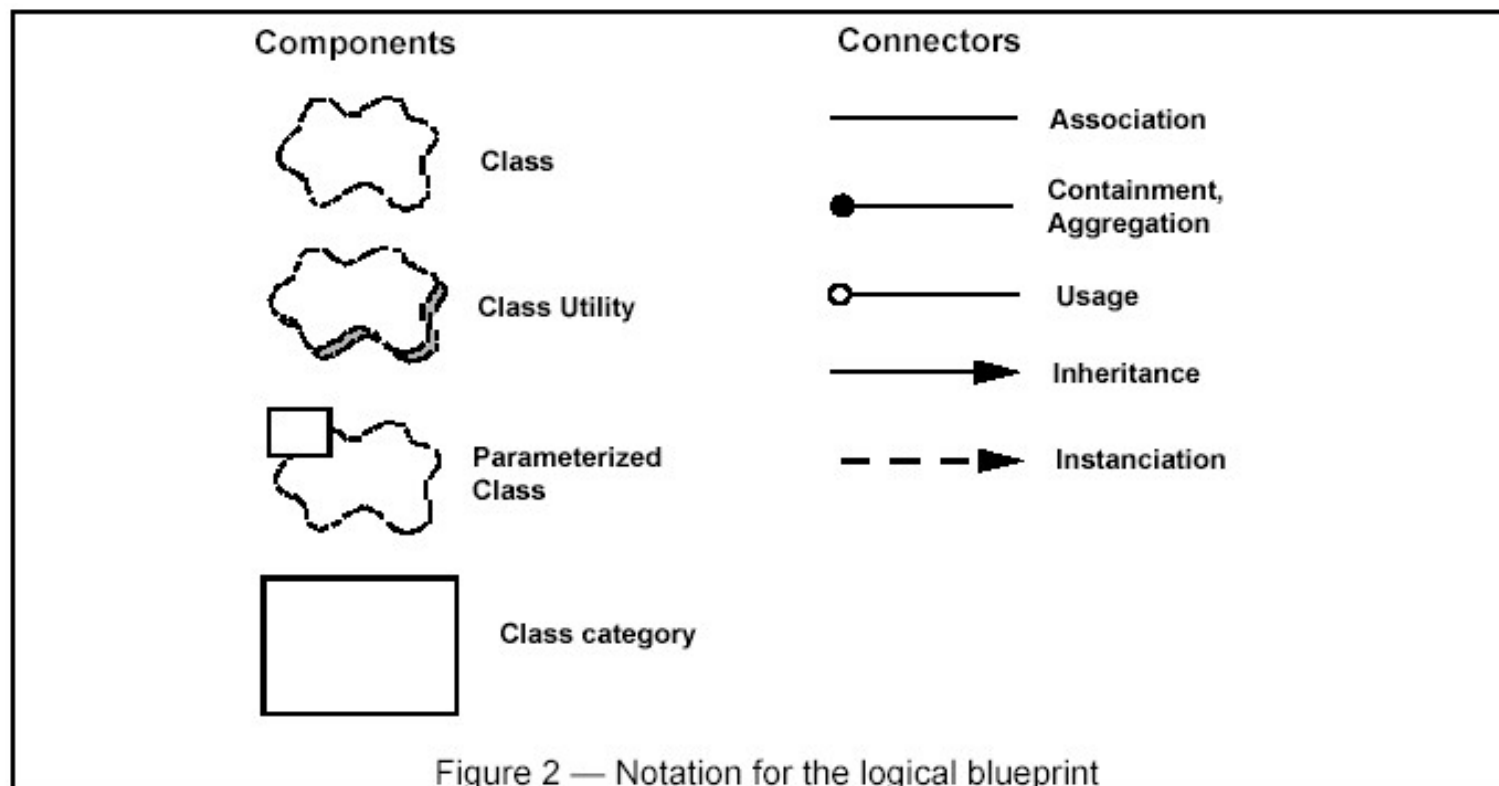


逻辑视图的体系结构：面向对象的分解

- 逻辑视图主要支持功能需求——系统应当向用户提供什么样的服务。从问题域出发，采用面向对象的方法，按照抽象、封装、继承的原则，进行分解，得到代表着系统的关键抽象表示的集合。这些抽象表示的具体形式就是对象和对象的类。这种分级不仅是为了功能分析，而且担负着在系统的各部分中确定公共机制和设计元素的作用。
- 使用Rational / Booch方法，通过类图(class diagram)和类模板(class template)来表示逻辑体系结构。类图显示了类的集合和它们的逻辑关系:关联(association)、组合 (composition)、使用(usage)、继承 (inheritance)等。类模板则着眼于每个类的个体，强调类的主要操作，并确定对象的关键特征。当十分需要定义一个对象的内部行为时，要使用状态转换图(state transition diagram)，或者是状态表(state chart)。相关类的集合可以归到一起，称作类的种属(class category)。

逻辑视图的符号表示法

逻辑体系结构的符号表示法(见图), 是从Booch方法派生而来的。它被极大地简化了, 尤其大量简化了在这个设计阶段作用不大的各种修饰, 只考虑对于体系结构有重要意义的元素在设计工具上, 可以使用Rational Rose等UML建模工具。公共的机制和服务在类设施 (class utilities)中定义。



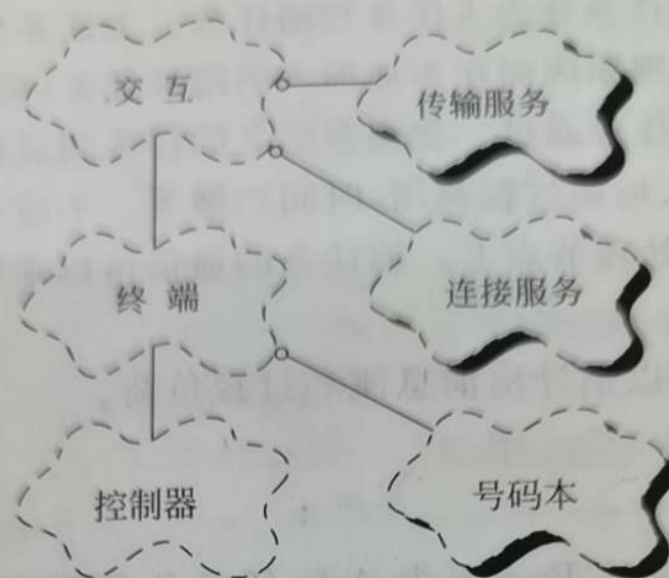
逻辑视图的风格

逻辑视图也可以采用面向对象的风格。

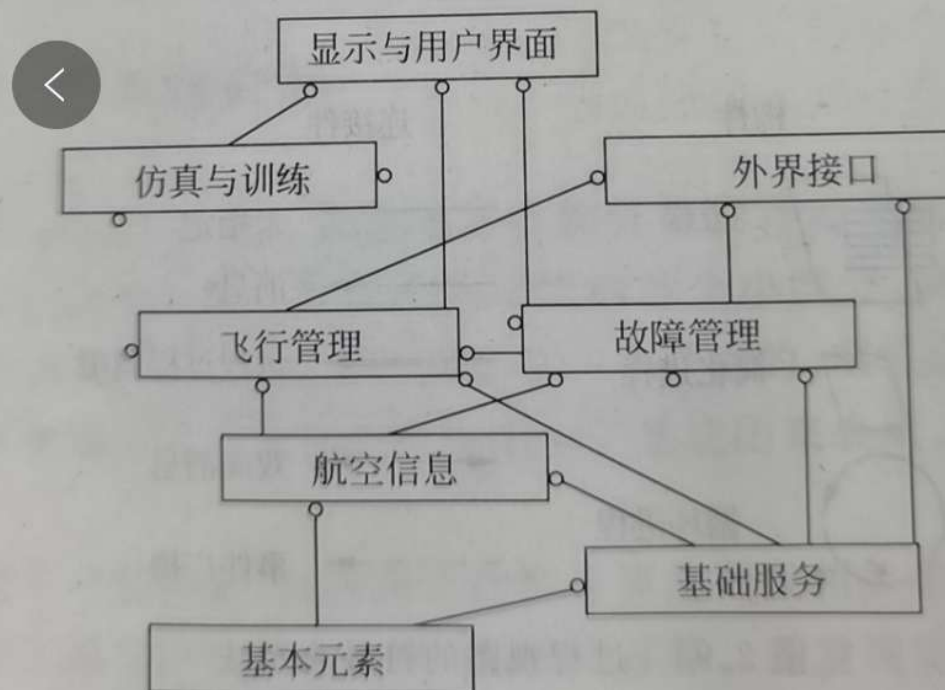
逻辑视图设计的主要准则是，要设法在整个系统中保持一个单一的、连贯的对象模型，避免类和相关机制出现按照场地或处理器过早的分化。

逻辑视图的例子

原文中显示了一个专用自动交换分机的例子。专用自动交换分机用于在通信终端之间建立连接。通信终端可能是电话机、中继线(连接到中心室的线路)、专用线(专用自动交换分机和一般的交换分机之间的线路)、数据线、**ISDN**线等。不同的线路需要不同的线路接口卡的支持。线路控制器对象负责从线路接口卡接受信号，以及向它发送信号，并完成信号和一系列的事件(如开始、结束、计数等)之间的转换。控制器还必须受到严格的实时要求的约束。为了适应不同的接口，这个类有许多子类。终端对象负责维护终端的状态，并代表所在的线路提供通信服务。对话对象代表在一个对话中涉及的终端的集合。对话对象使用转换任务(逻辑地址和物理地址之间的映射、路由等)和连接服务建立两个终端之间的语音连接。



(a) 专用自动交换分机的逻辑设计图



(b) 航空交通管制系统的逻辑设计



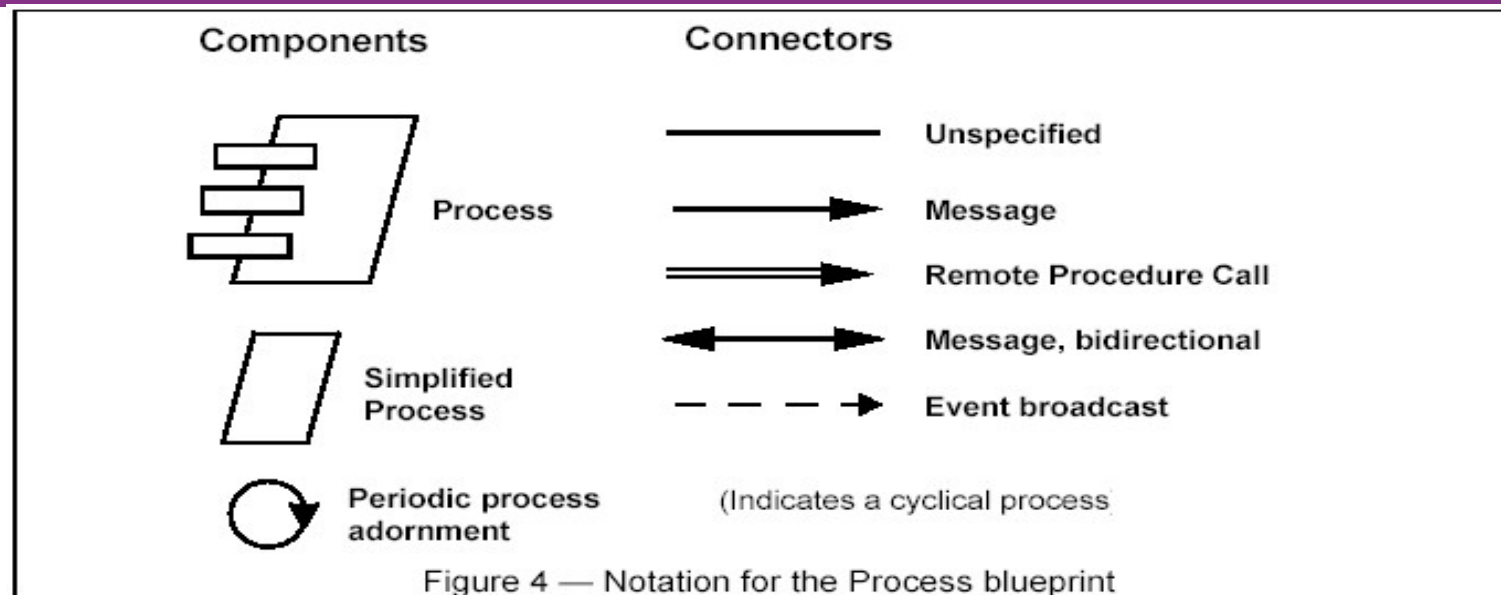
过程视图的体系结构：过程分解

- 过程体系结构考虑的是一些非功能性的需求，诸如性能、可用性等。它所面对的问题有并发，分布，系统的完整性，容错能力等。它还要考虑怎样把过程体系结构与逻辑视图体系结构的要点相适应——对某个对象的某个操作实际上是在哪个控制线程上发生的。
- 可以把过程体系结构分为几个抽象层次来描述，每个层次考虑不同的方面。在最高层次上，过程体系结构可以被视为是一个逻辑网络的集合。每个独立执行的逻辑网络都是由通信程序(即“过程”)构成的。这些逻辑网络分布在一个通过LAN或WAN连接起来的硬件资源集合上。多个逻辑网络可能同时存在，并共享同样的物理资源。例如，逻辑网络的概念可用于区分在线处理系统和离线处理系统。



过程视图的体系结构：过程分解

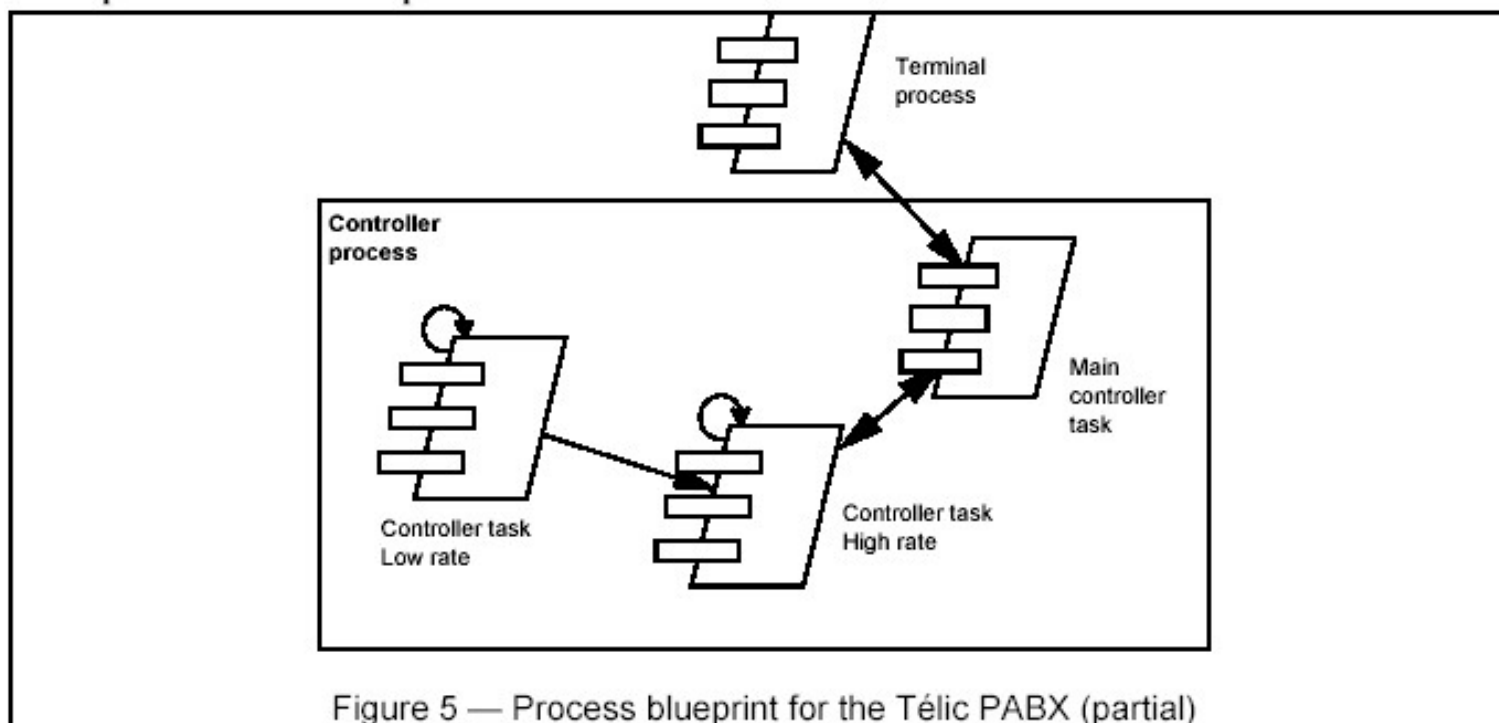
- 软件被分为独立的任务的集合。每个任务是一个独立的控制线程，可以在一个处理节点上独立单独调度。因此可以将任务分为主任务和辅任务。主任务是需要单独解决的体系结构元素。辅任务是由于实现原因而在本地加入的附加任务(缓冲，超时，等等)，例如可以将它们实现为轻量级的线程。主任务通过一套完善定义的任务间通信机制进行通信：同步的或异步的基于消息的通信服务、远程过程调用、时间广播等。不应当假设通信中的主任务处于同一个过程中或处在同一个处理节点上。辅任务的通信可以采用共享内存的方式或其他双方约定的方式。
- 基于过程体系结构设计图，可以估计出消息流和过程负荷。



在辅助工具的选择上，可以考虑使用TRW提供的UNAS (Universal Network Architecture Services) 产品。它可用于把各种过程和任务构建并实现为过程的逻辑网络。UNAS里面包含的一个工具SALE (Software Architecture Lifecycle Environment) 支持这样的符号表示法。SALE允许过程体系结构的图形化描述，包括对可能的任务间通信路径的规格说明。然后，从这种规格说明可以自动生成相应的Ada或C++语言源代码。

有多种风格适合过程体系结构。例如管道和过滤器、客户 / 服务器及其变体(多客户 / 单服务器, 多客户 / 多服务器)等。

Example of a Process blueprint



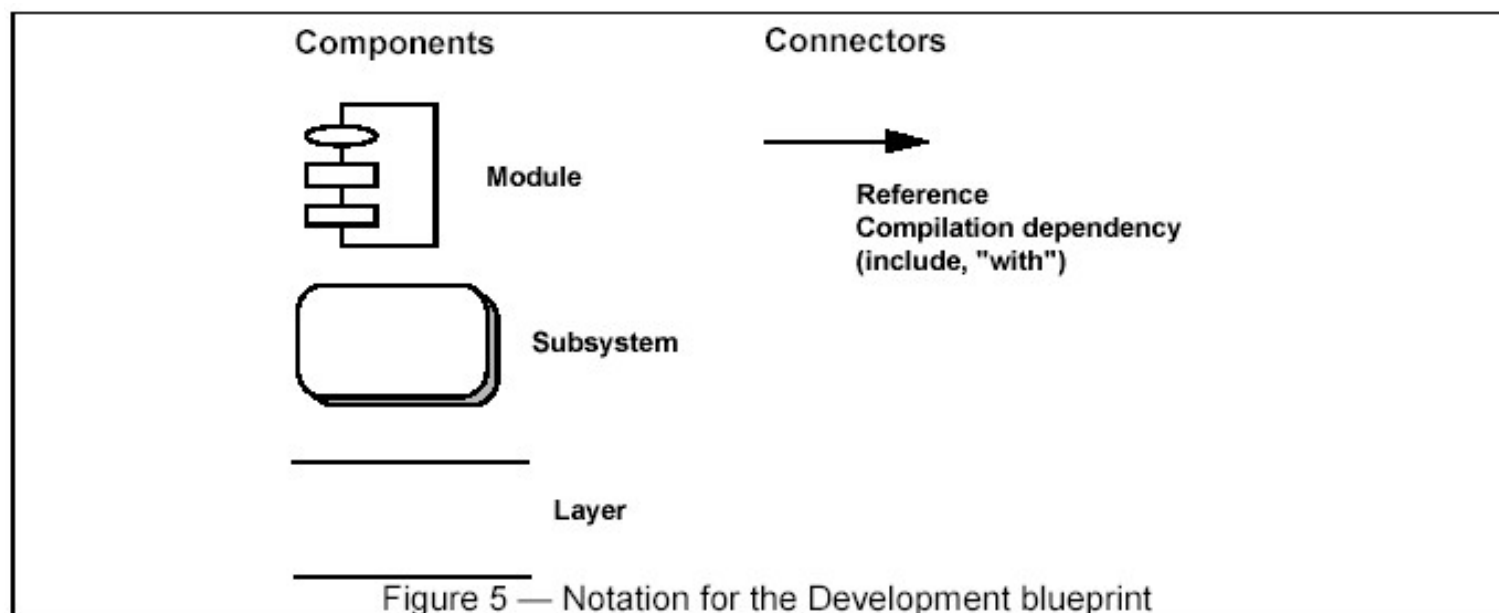
开发视图的体系结构：子系统分解



- 1) 开发视图，关注的是在软件开发环境中软件模块的实际组织。软件被打包成可以由单个或少量程序员开发的各种小的部分：程序库或子系统。子系统被组织成层次化的体系，每一层为上一层提供一个严密的、明确定义的接口。
- 2) 系统的开发体系结构用模块图和子系统图来表示，在图中可以显示出“导入”和“导出”关系。完整的开发体系结构只有在软件系统的所有元素被识别出来之后才能被描述。控制开发体系结构的原则是：分割、编组、可视。
- 3) 开发体系结构主要考虑的是内部需求，这些需求目的是要使开发相关的活动更易于进行，如软件管理、软件复用、开发工具集所造成的约束、编程语言等。开发体系结构是许多开发活动的基础，包括需求配置、团队组织和工作分配、成本估算和成本规划、项目进度监控、软件可重用性和可移植性分析、软件安全分析等。它是建立软件产品线的基础。

开发视图的符号表示法

与前面类似，开发视图的符号表示法采用Booch表示法的变体，并且只考虑对于体系结构有重要意义的元素，如图所示。在Rational Rose中，可以绘制模块层和子系统层的开发体系结构图，还可以在逆向工程中从已经开发的源代码(Ada或C++)得出系统的开发体系结构图。





开发视图的风格

对于开发视图，我们建议采用分层风格，定义4—6层的子系统。每一层都有明确责任。设计规则是，某一层的子系统只能依赖于本层或其下层的子系统。这样做的目模块间相互依赖而构成的复杂网络最小化，并使得系统可以采用逐层的策略完成释放。

开发视图的例子

下图用5个层次表示了航空交通管制系统产品线的开发组织。此开发体系结构图3-4 (b) 中描述的逻辑体系结构相对应的。

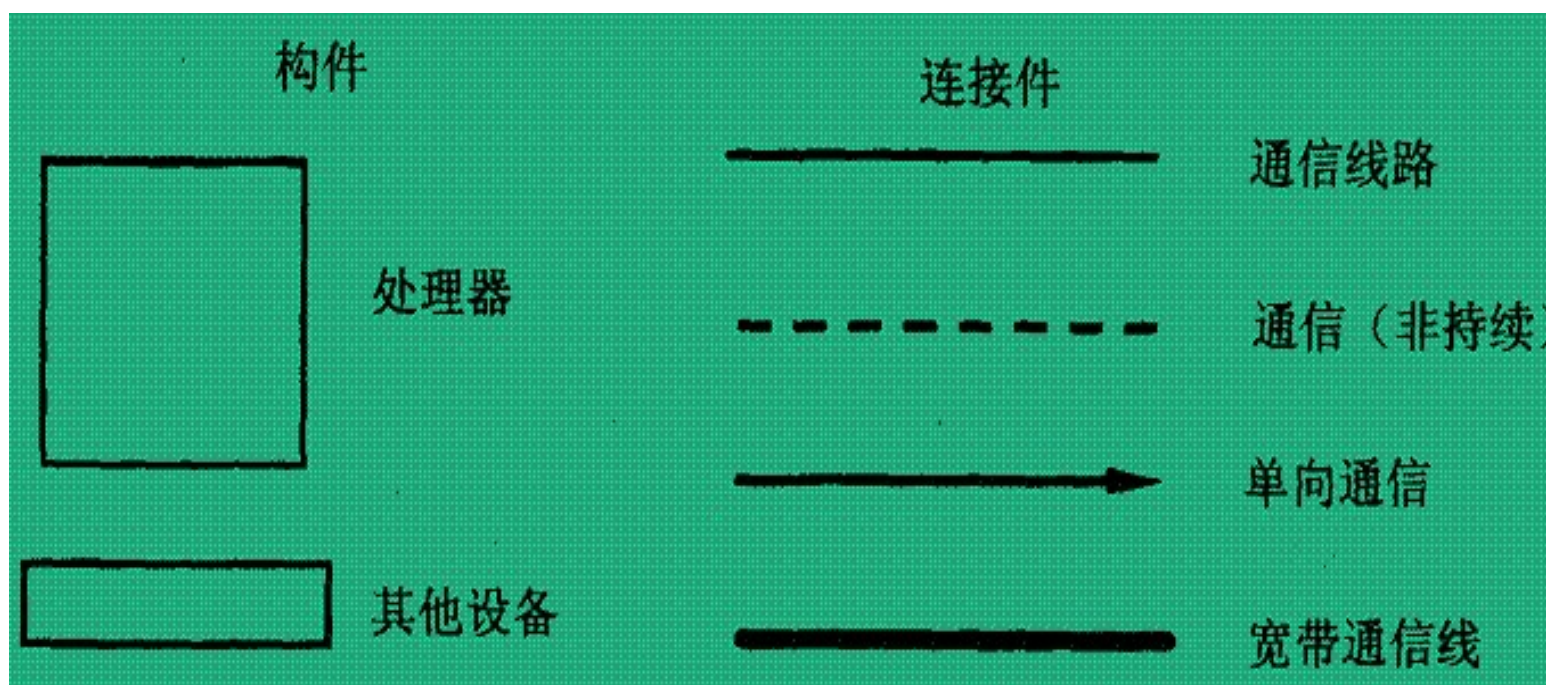


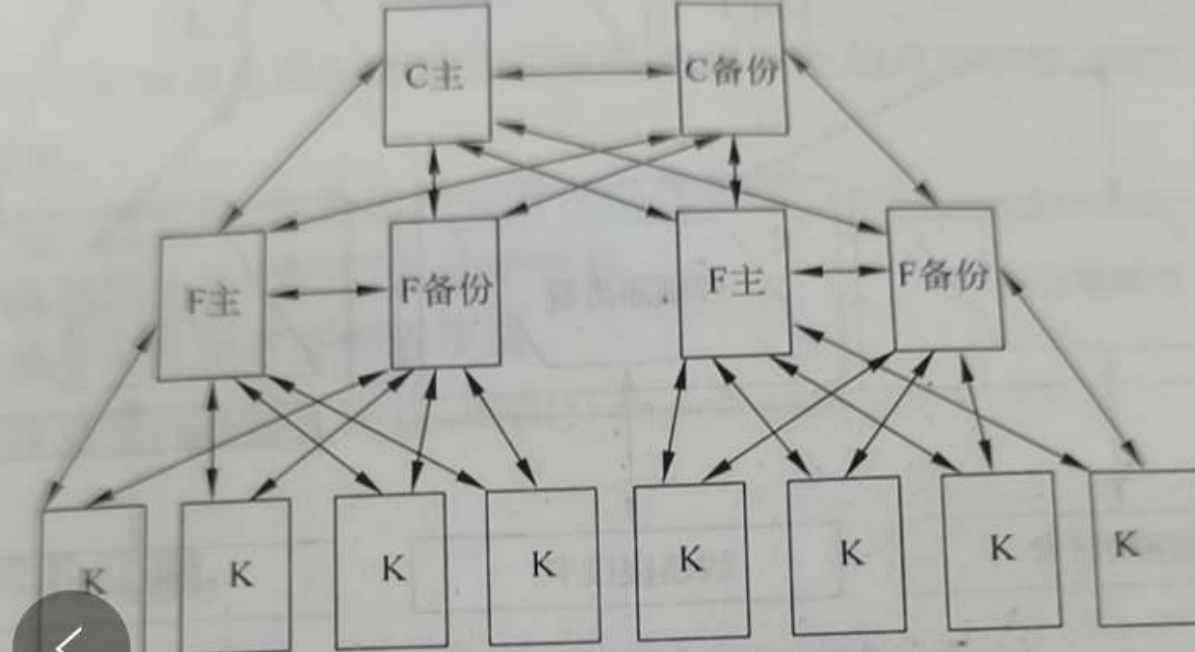


物理体系结构主要考虑的是非功能性的系统需求，如系统的可用性、可靠性(容错性)、性能(信息吞吐量)和可扩展性。软件系统在计算机网络的各个处理节点上运行。各种被确定出的元素——网络、过程、任务和对象——需要映射到各种节点上去。将用到不同的物理配置。有些用于开发和测试，有些用于不同场所或不同用户。因此从软件到处理节点的映射需要高度灵活，并且最小限度地影响其本身的源代码。

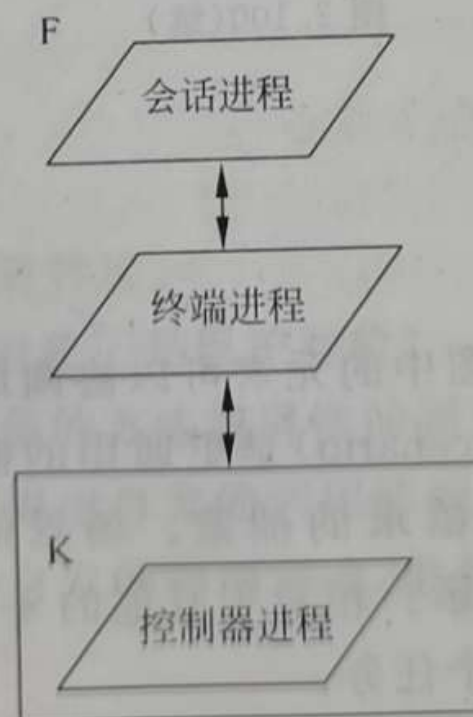
物理视图的符号表示法

TRW公司的UNAS允许使用者采用数据驱动的方式将过程体系结构映射到物理体系结构，并允许在不修改源代码的情况下对这种映射做出多种改动。





(a) ACS系统的物理视图



(b) 具有进程分配的小型ACS系统的物理视图

场景视图的体系结构：汇总

通过使用一些重要场景，4个视图中的元素可以协调地共同工作。尽管这些场景是一个小集合，但是它们很重要。场景(scenario)是更通用的概念——用例(use case)的实例。从某种意义上讲，场景是最重要的需求的抽象。场景的设计使用对象场景图(object scenario diagram)和对象交互图来表示。

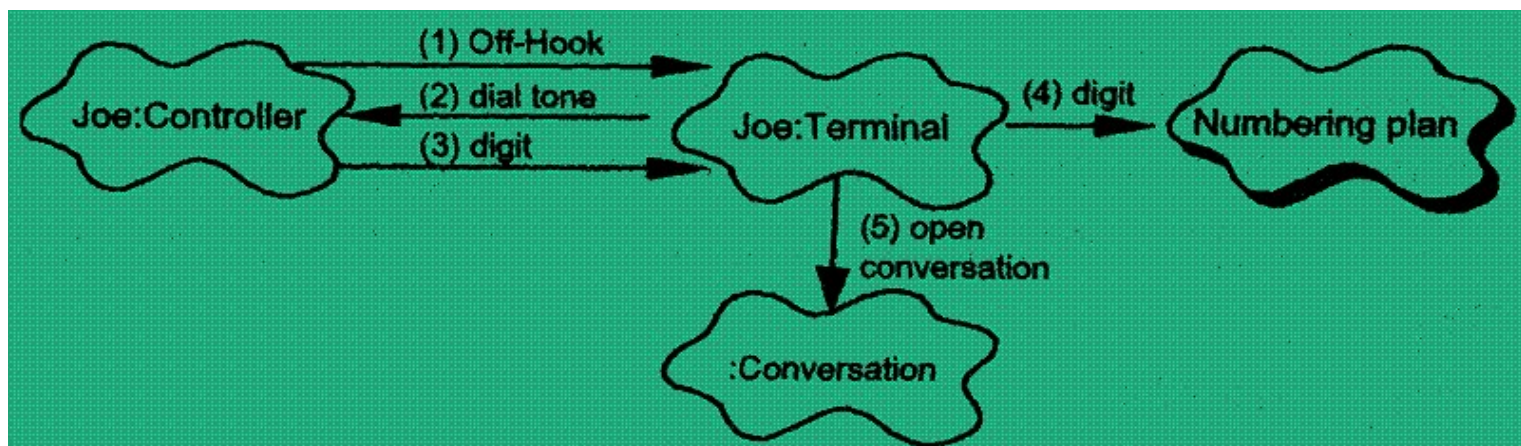
相对于其他的4个视图，这个视图是多余出来的(所以称为“4+1”)，但是它承担着两个目的：

- 在体系结构设计中，将以此视图为驱动来发现体系结构元素。
- 在体系结构设计结束后，此视图承担验证和描述的角色。它不仅用于书面记录，并且是体系结构原型测试的起始点。



场景视图的符号表示法

场景视图的符号表示法中，构件的表示与逻辑视图非常相似，但是连接件的表示使用过程视图中的方法。注意，对象的实例用细实线表示。在工具的使用方面，和在逻辑体系结构类似，可以使用Rational Rose绘制和管理对象场景图。



场景的雏形—本地呼叫选择阶段

- ① Joe的电话的控制器检测到并证实了从挂起到取下的状态转变，并且发送了消息来唤醒相关的终端对象。
- ② 终端分配一些资源，并告诉控制器发出拨号音。
- ③ 控制器受到数字并将它们发送给终端。
- ④ 终端使用号码规划来分析数字流。
- ⑤ 当一个有效序列的输入完成，终端打开一个对话。